

Onboard NN and RL Flight Summary

Only flights in which the NN or RL controller was actually activated are included. In this document, p , q , and r are the body angular rates. Large oscillations in these signals indicate vibration or closed-loop instability. The x/y errors are horizontal-position errors, while z is the altitude error.

Main Findings

- The NN models generally became better at reducing horizontal error, but most still lost altitude while doing so.
- A 10 Hz low-pass filter reduced high-frequency noise in p , q , and r , but did not remove slower oscillations or solve the altitude-control problem.
- The best RL result came from `recurrent_ppo_figure8_gates_correct_yaw_tau_001_noise_all_96000000_steps.zip`. It operated for a comparatively long time, but the drone eventually reached the floor and crashed.
- The dominant limitation of both controller families was vertical control. Several models moved the drone towards the horizontal target but did not command enough stable thrust to maintain the requested altitude.

Flight Configuration

OptiTrack

The earlier flights used OptiTrack position updates at 10 Hz. The neural controller itself ran at approximately 100 Hz, so several controller executions could occur between new OptiTrack measurements.

The later flights used the highest OptiTrack publishing frequency available. This provided fresher position data, but did not by itself solve the instability or altitude loss.

Filtering of p , q , and r

Most flights used the raw angular-rate inputs without an onboard filter. Some later experiments used a first-order low-pass filter with a 10 Hz cutoff.

The filter attenuated the visible high-frequency vibration, especially in p and q . It did not eliminate the lower-frequency motion generated by the controller. Consequently, filtered flights could still oscillate, descend, and crash. The 25 Hz filters visible in some old analysis plots were applied after the flights and were not active onboard.

Measured Rotor Noise

Before one RL activation, the standard deviations of the measured angular rates were approximately 0.209 rad/s in p , 0.078 rad/s in q , and 0.082 rad/s in r . A dominant component appeared near 32–33 Hz, consistent with rotor vibration as represented at the controller sampling rate.

After unstable controllers were activated, the angular-rate excursions became much larger than this initial noise. The crashes therefore cannot be explained by sensor noise alone; the closed loop was also becoming unstable.

External-Moment (M_{ext}) Ranges

The ranges below were calculated only from samples in which the NN controller was active. Values are in N m. The observer defines the estimated external moment as the measured moment minus the modeled moment.

Before NN Activation

The onboard M_{ext} fields were all zero between takeoff and NN activation, even when the configured observer mode was **BACKGROUND** or **ON**. Inspection of the firmware showed that the observer was only executed inside the active NN control routine. These zeros are therefore placeholders, not measurements of zero physical moment.

To estimate the pre-activation moment, the observer was replayed offline using the logged body rates, velocity, attitude, and measured motor RPM. The same 8 Hz second-order Butterworth filter, inertia values, aerodynamic model, and moment equation used onboard were applied. Over the final five seconds of stabilized flight before NN activation, combining the four available NN flights, the reconstructed values were:

$M_{ext,x}$: -0.014623 to 0.008496 N m, with a mean of -0.002660 N m.

$M_{ext,y}$: -0.014009 to 0.004661 N m, with a mean of -0.005453 N m.

$M_{ext,z}$: -0.008576 to 0.008470 N m, with a mean of 0.000507 N m.

These reconstructed pre-activation moments were much smaller than the extreme values observed after NN activation. They should be treated as approximate because they were recalculated from logged samples rather than recorded directly from a continuously running onboard observer.

During NN Activation

Considering every NN-active sample for which the observer was running, the estimated external moments were:

$M_{ext,x}$: -0.363538 to 0.363531 N m.

$M_{ext,y}$: -0.306448 to 0.386990 N m.

$M_{ext,z}$: -0.521556 to 0.436255 N m.

With the observer in **BACKGROUND** mode, the estimate was calculated and logged but was not supplied to the NN. The ranges were $M_{ext,x} = -0.363538$ to 0.363531 N m, $M_{ext,y} = -0.306448$ to 0.333567 N m, and $M_{ext,z} = -0.521556$ to 0.436255 N m.

With the observer **ON**, the estimate was supplied to the NN. The ranges were $M_{ext,x} = -0.270788$ to 0.345831 N m, $M_{ext,y} = -0.268361$ to 0.386990 N m, and $M_{ext,z} = -0.165128$ to 0.155991 N m.

Across all NN-active observer samples, the measured-moment ranges were -0.354669 to 0.345692 N m in x , -0.306300 to 0.388041 N m in y , and -0.520981 to 0.436139 N m in z . The corresponding modeled-moment ranges were -0.290800 to 0.320735 N m in x , -0.241375 to 0.250627 N m in y , and -0.030120 to 0.016223 N m in z .

During the RL flights that contained M_{ext} fields, the observer mode was **OFF**. All logged M_{ext} fields were therefore zero by configuration; this does not mean that the physical external moment was zero.

NN Models

Early 64-unit CfC model

Most likely model: `new_CfC_64_neurons_seq_1_epoch=18_val_loss=0.000142.ckpt`

This model produced the longest early NN flights. In two activations, the drone remained airborne for approximately 51 and 24 seconds. It maintained an altitude around 1.0–1.2 m, but oscillated strongly and drifted by more than two metres horizontally. It could keep the drone in the air, but did not provide accurate position control.

In another short activation, the horizontal error—especially in y —decreased considerably while the drone started to lose altitude. This was an early example of improved x/y behavior without adequate z control. These flights used 10 Hz OptiTrack updates and no confirmed onboard $p/q/r$ filter.

Early Bebop2 CfC model

Most likely model: `bbp2_conv_cfc_default_n64_epoch=19_val_loss=0.000143.ckpt`

This model was unstable in the available onboard activations. The drone crashed roughly 1.3–1.4 seconds after activation. Some flights were dominated by large p oscillations, while another showed particularly strong motion in q . There was not enough stable flight time for the position error to converge. These flights used 10 Hz OptiTrack updates and no confirmed onboard $p/q/r$ filter.

No-dt LeCun model

Model: `conv_cfc_default_n64_bebop2_no_dt_lecun.ckpt`

This NN did not receive an explicit time-step input and used a 10 Hz low-pass filter on $p/q/r$. OptiTrack was also limited to 10 Hz. The drone remained airborne for about five seconds. Its horizontal error decreased substantially, but its altitude error increased continuously until it crashed. Filtering reduced part of the fast angular-rate vibration, but did not prevent vertical loss or the final instability.

Delayed correct-sign model

Model: `conv_cfc_default_n64_bebop2_delayed_correct_sign.ckpt`

This was the main later NN model. It did not receive dt explicitly and relied on the CfC's implicit temporal behavior. These flights used the maximum OptiTrack publishing frequency.

Without the $p/q/r$ filter, the model often approached the horizontal target but gradually descended. Depending on the activation, it remained airborne for about 8–17 seconds. Some runs were stopped while the drone was still flying low; others ended on the floor. The dominant oscillation varied between p and r .

With the 10 Hz filter, the high-frequency component became visibly smaller and the model still reduced the x or x/y error. However, the drone continued to oscillate laterally and lose altitude. One filtered activation lasted about 17 seconds before reaching the floor, while other activations crashed much sooner because of large or low-frequency p/q motion.

The model was also tested with an external-moment observer. In **BACKGROUND** mode, the moment was estimated and logged but was not supplied to the NN. In **ON** mode, the estimate was included in the NN input. The single flight with the observer ON still showed strong q oscillation, reasonable horizontal behavior, and rapid altitude loss. There is not enough data to isolate the observer's effect from the other flight-to-flight differences.

Bebop2 no-dt-delayed variant

Model label: bebop2-no-dt-delayed

This variant kept x/y relatively close to the target for approximately 16 seconds, but progressively lost altitude and crashed. The largest angular-rate oscillation was in r . OptiTrack was at its maximum publishing frequency and no $p/q/r$ filter was used.

RL Models

Early recurrent PPO family

Model family: recurrent_ppo_figure8_gates.zip; exact checkpoint unavailable.

With 10 Hz OptiTrack and no onboard $p/q/r$ filter, the drone began moving towards the horizontal target but lost altitude and crashed after approximately 2–3 seconds.

Recurrent PPO with plus-one convention

Model: recurrent_ppo_with_plus1/recurrent_ppo_figure8_gates_ckpt_44800000_steps.zip

This model was tested with both filtered and unfiltered angular rates while OptiTrack was limited to 10 Hz. With the 10 Hz filter, the drone moved towards the target in one horizontal direction, overshoot in the other, lost altitude, and crashed after about four seconds. Without the filter, it crashed after about two seconds with substantial y overshoot and rapid altitude loss. Removing the filter did not improve the result.

83.2M-step recurrent PPO

Model: recurrent_ppo_figure8_gates_83200000_steps.zip

With 10 Hz filtering and 10 Hz OptiTrack, this model partially reduced the horizontal error but failed to maintain altitude. The drone crashed after approximately two seconds.

No-noise plus-one export

Model: RL_CFC_no_noise_plus1, 44.8M-step export.

This model used maximum-rate OptiTrack and no $p/q/r$ filter. It made a small horizontal correction, but the commands and q became highly oscillatory. The drone lost altitude and crashed in under two seconds.

Measurement-noise export

Model: RL_CFC_noise_measurements, 51.2M-step export.

This model used maximum-rate OptiTrack and no onboard $p/q/r$ filter. The y error decreased substantially, but x did not improve and the altitude error increased. The drone crashed after approximately three seconds. Adding measurement noise during training was therefore not sufficient to stabilize the real flight.

All-noise 96M model

Model: recurrent_ppo_figure8_gates_correct_yaw_tau_001_noise_all_96000000_steps.zip

This model was trained with noise applied to all selected randomized quantities. It used maximum-rate OptiTrack, no onboard $p/q/r$ filter, and no external-moment input.

This was the best RL model in the onboard tests. It ran for approximately ten seconds, considerably longer than most other useful RL activations. The flight still showed horizontal oscillation and delay, and the drone fell to the floor at the end.

All-noise 57.6M model

Model: recurrent_ppo_figure8_gates_correct_yaw_tau_001_noise_all_57600000_steps.zip

This model used maximum-rate OptiTrack, no onboard $p/q/r$ filter, and no external-moment input. The activation was short and unstable. The drone developed a large q excursion, failed to reach the target, lost altitude, and crashed quickly.

Second p/q/r-noise 51.2M model

Model: recurrent_ppo_figure8_gates_correct_yaw_tau_001_noise_pqr2_51200000_steps.zip

This model used maximum-rate OptiTrack, no onboard $p/q/r$ filter, and no external-moment input. In one activation, it substantially reduced the horizontal error, but the altitude error increased and the drone reached the floor. In another, it improved x/y during part of the flight and initially kept altitude reasonably close to the target, but then developed oscillations, lost altitude, and crashed. These results were separate from the better all-noise 96M flight.

Overall Assessment

The NN and RL results show the same broad limitation. Horizontal control improved enough for several models to reduce x/y error, but the correction was often accompanied by oscillation and a loss of altitude. Increasing the OptiTrack update rate and filtering $p/q/r$ improved the quality of the inputs, but neither change resolved the main control problem.

The next tests should focus on the vertical channel: the sign and normalization of z and vertical velocity, the conversion from network output to motor RPM, the real hover RPM, motor delay, and the modeled thrust response. Fixed-target hover tests would make these effects easier to separate before returning to multi-target trajectories.